How the Internet Delivers a Web Page

A Case Study of www.amazon.com

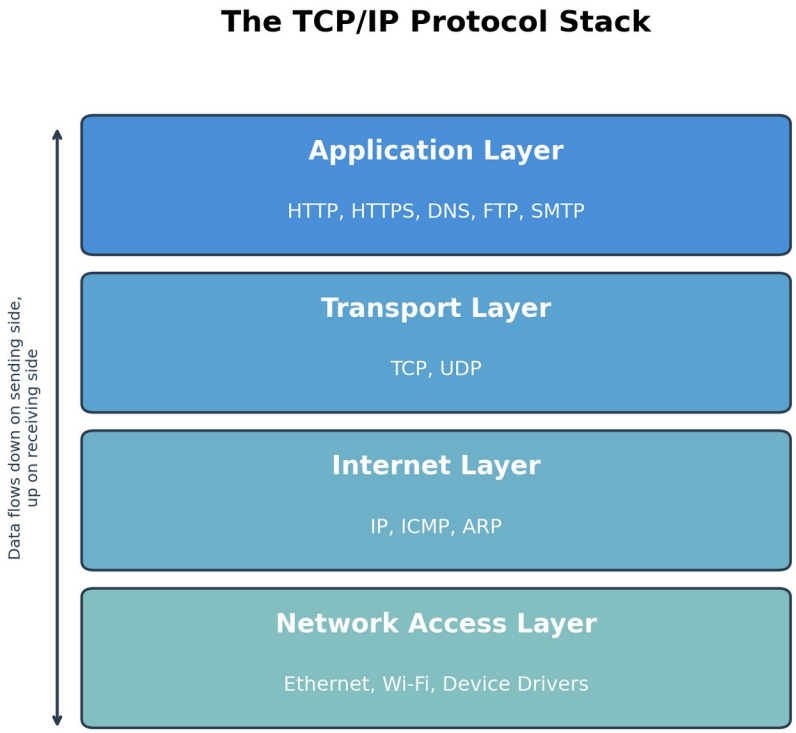
Assignment Report

Introduction

Every time a person opens a browser, types a web address such as www.amazon.com, and presses Enter, a complex sequence of events takes place within a fraction of a second. This report uses Amazon's website as a case study to explain, step by step, how the internet's protocol stack works together to locate, request, and deliver a web page to the user's screen. It covers the TCP/IP protocol stack, the role of DNS, how HTTP and TCP cooperate to deliver reliable content, the client-server architecture of the application, a comparison between HTTP and FTP, the role of cookies, and a complete flow diagram of the communication process.

1. The TCP/IP Protocol Stack

The TCP/IP model organizes network communication into four layers. Each layer has a specific responsibility, and data passes down through the layers on the sending side (encapsulation) and up through the layers on the receiving side (decapsulation).



Application Layer

This is the topmost layer, closest to the user. It contains protocols such as HTTP/HTTPS, DNS, FTP, and SMTP that applications use directly. When a person visits Amazon, the browser uses HTTP/HTTPS at this layer to format the request for the web page, and DNS to resolve the domain name.

Transport Layer

This layer is responsible for end-to-end communication between the client and the server. It uses TCP, which provides reliable, ordered, error-checked delivery of data (used for loading web pages), or UDP, which is faster but does not guarantee delivery (used for services like video streaming or DNS lookups). For a website such as Amazon, TCP is used to guarantee that all parts of the web page arrive intact and in order.

Internet Layer

This layer handles logical addressing and routing of data across networks. The IP protocol assigns addresses (such as Amazon's server IP) and determines the best path for data packets to travel from the client's device to the destination server, potentially crossing many intermediate routers.

Network Access Layer

Also called the link layer, this is the lowest layer and deals with the physical transmission of data over a specific medium, such as Ethernet cables or Wi-Fi. It handles hardware addressing (MAC addresses) and the actual electrical, radio, or optical signals that carry the bits from one device to the next hop on the network.

2. Role of DNS in Converting Domain Names to IP Addresses

Computers communicate using numerical IP addresses (for example, 205.251.242.103), not human-friendly names like www.amazon.com. The Domain Name System (DNS) acts as the internet's phonebook, translating domain names into IP addresses so that the browser knows where to send its request.

When the user types www.amazon.com and presses Enter, the process typically follows these steps:

- The browser first checks its own cache, then the operating system's cache, for a previously resolved IP address.
- If not found, the request goes to a DNS resolver (often provided by the ISP), which queries a hierarchy of DNS servers: root servers, then the .com top-level domain (TLD) servers, then Amazon's authoritative name servers.
- Amazon's authoritative DNS server returns the correct IP address for its web server (or, more commonly, for a load-balancer or content delivery network edge server).
- The resolved IP address is cached locally for a period of time (its Time-To-Live) so future requests to the same domain are faster.

Only once this IP address is known can the browser establish a connection to Amazon's server.

3. How HTTP Requests and Receives the Website

HTTP (HyperText Transfer Protocol), or its secure version HTTPS, is the application-layer protocol used to request and deliver web content. Once the browser has Amazon's IP address and a TCP connection is open, it uses HTTP to communicate with the web server as follows:

Request: The browser sends an HTTP GET request to the server. This request includes the requested resource path (e.g. "/"), the HTTP method, headers (such as browser type, accepted content formats, and cookies), and the target host name.

Processing: Amazon's web server receives the request, processes it (possibly querying databases for product listings, personalized recommendations, and account data), and prepares an HTTP response.

Response: The server sends back an HTTP response containing a status code (such as 200 OK), response headers, and the requested content, typically an HTML document.

Additional requests: The browser parses the HTML and discovers references to further resources, such as CSS stylesheets, JavaScript files, and images. It sends additional HTTP requests for each of these, which are also returned by the server.

Rendering: The browser assembles all the received resources and renders the complete Amazon web page on the user's screen.

4. How TCP Provides Reliable Communication for HTTP

HTTP relies on TCP (Transmission Control Protocol) at the transport layer to ensure that data is delivered completely, correctly, and in the right order. TCP provides reliability through several mechanisms:

- Connection establishment: Before any HTTP data is exchanged, TCP performs a three-way handshake (SYN, SYN-ACK, ACK) between the browser and Amazon's server to establish a reliable connection.
- Sequencing: Data is broken into segments, each labeled with a sequence number, so the receiving end can reassemble them in the correct order even if they arrive out of order.
- Acknowledgements and retransmission: The receiver acknowledges received segments; if an acknowledgement is not received within a certain time, TCP retransmits the lost segment, ensuring no data is silently dropped.
- Flow control and congestion control: TCP adjusts the rate of data transmission based on the receiver's buffer capacity and network congestion, preventing data loss due to overload.
- Error checking: Checksums in each TCP segment allow the receiver to detect corrupted data and request retransmission.
- Connection termination: Once the full page and its resources have been transferred, the TCP connection is closed gracefully using a FIN/ACK exchange.

Because of these guarantees, HTTP itself does not need to worry about lost or out-of-order packets; it can assume that whatever TCP delivers to it is complete and correctly ordered.

5. Client-Server or Peer-to-Peer Architecture?

Amazon's website follows a client-server architecture, not a peer-to-peer one. In this model, the user's browser acts as the client, which sends requests, and Amazon's infrastructure (web servers, application servers, databases, and content delivery networks) acts as the server, which processes requests and returns responses.

This classification is justified for several reasons:

- Centralized control: All product data, pricing, inventory, and account information are stored and managed centrally by Amazon's servers, not distributed among individual users' devices.
- Asymmetric roles: The browser (client) never serves content to other browsers directly; it only consumes services from Amazon's servers. In a true peer-to-peer system, each node would act as both client and server.
- Scalability and security requirements: E-commerce platforms need centralized control over transactions, payments, and security, which client-server architecture supports far better than peer-to-peer, where trust and consistency are harder to guarantee.
- Session and state management: Amazon needs to reliably track shopping carts, user accounts, and orders on the server side, which requires a central authority rather than a distributed peer network.

6. Comparison of HTTP and FTP

Although both HTTP and FTP are application-layer protocols used to transfer data over the internet, they differ significantly in purpose and design. The table below highlights at least four key differences.

Feature	**HTTP**	**FTP**
Purpose	Transfers web pages and resources (HTML, CSS, images) for display in a browser	Transfers files between a client and server, often for upload/download of whole files
Connections used	Typically uses a single TCP connection	Uses two separate TCP connections: a

	(port 80/443) for requests and responses	control connection (port 21) and a data connection (port 20)
Statefulness	Stateless by design; each request is independent (cookies/sessions used to add state)	Stateful; the server keeps track of the client's session and current directory
Authentication	Often no login required for public pages; authentication handled at the application layer	Usually requires a username and password (or anonymous login) before file transfer
Typical use case	Browsing websites such as Amazon, submitting forms, loading APIs	Uploading/downloading large files to/from a file server

7. How Cookies Improve User Experience

Cookies are small pieces of data that a website's server sends to the browser, which the browser stores and sends back with future requests to the same site. Because HTTP is stateless (it does not remember previous interactions on its own), cookies allow websites to maintain context about a user across multiple requests and visits.

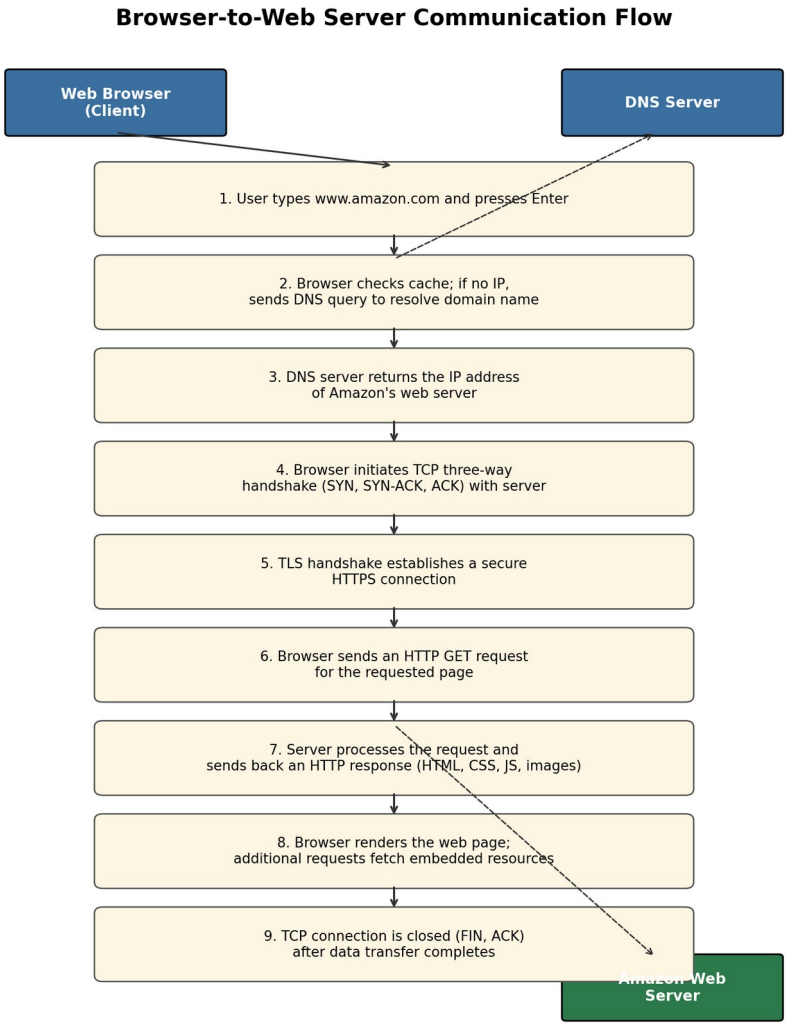
Cookies improve the user experience in several ways:

- Maintaining login sessions, so users are not asked to sign in on every single page.
- Remembering shopping cart contents between pages and visits.
- Storing user preferences, such as language, currency, or display settings.
- Enabling personalized recommendations based on browsing history.

Real-world example: On Amazon, when a user adds items to their shopping cart and then closes the browser without checking out, a cookie stores the cart's contents. When the user returns later, Amazon's server reads this cookie and restores the cart exactly as it was left, without requiring the user to search for and re-add each product. Cookies also allow Amazon to keep a user logged in across sessions and to show personalized product recommendations based on past browsing activity.

8. Complete Communication Flow: Browser to Web Server and Back

The diagram below summarizes the complete end-to-end communication process that takes place when a user visits www.amazon.com, from typing the URL to the page being fully rendered.



9. Conclusion

Application layer protocols such as HTTP, DNS, and FTP form the foundation of everyday internet usage, even though most users never see them directly. Every visit to a website like Amazon depends on DNS to translate a memorable name into a machine-readable address, HTTP to structure the request and response of content, and the underlying TCP/IP stack to guarantee that data travels reliably across a global network of routers and links. Without these standardized protocols working seamlessly together, browsing, shopping, streaming, and communicating online would be inconsistent, insecure, and unreliable. Understanding this process highlights how much coordinated engineering supports something as simple as loading a single web page, and underscores why application layer protocols remain central to the functioning of the modern internet.